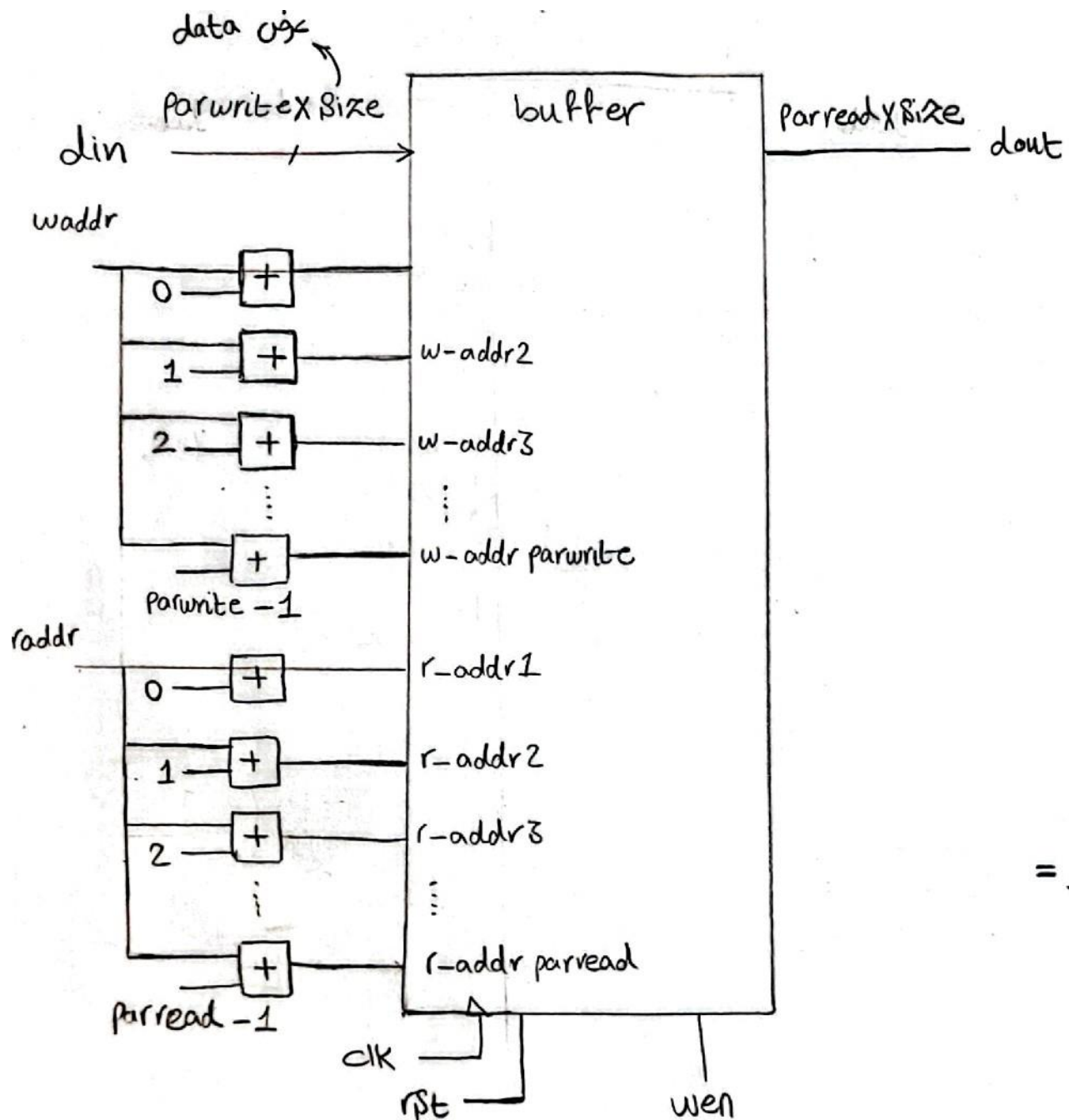
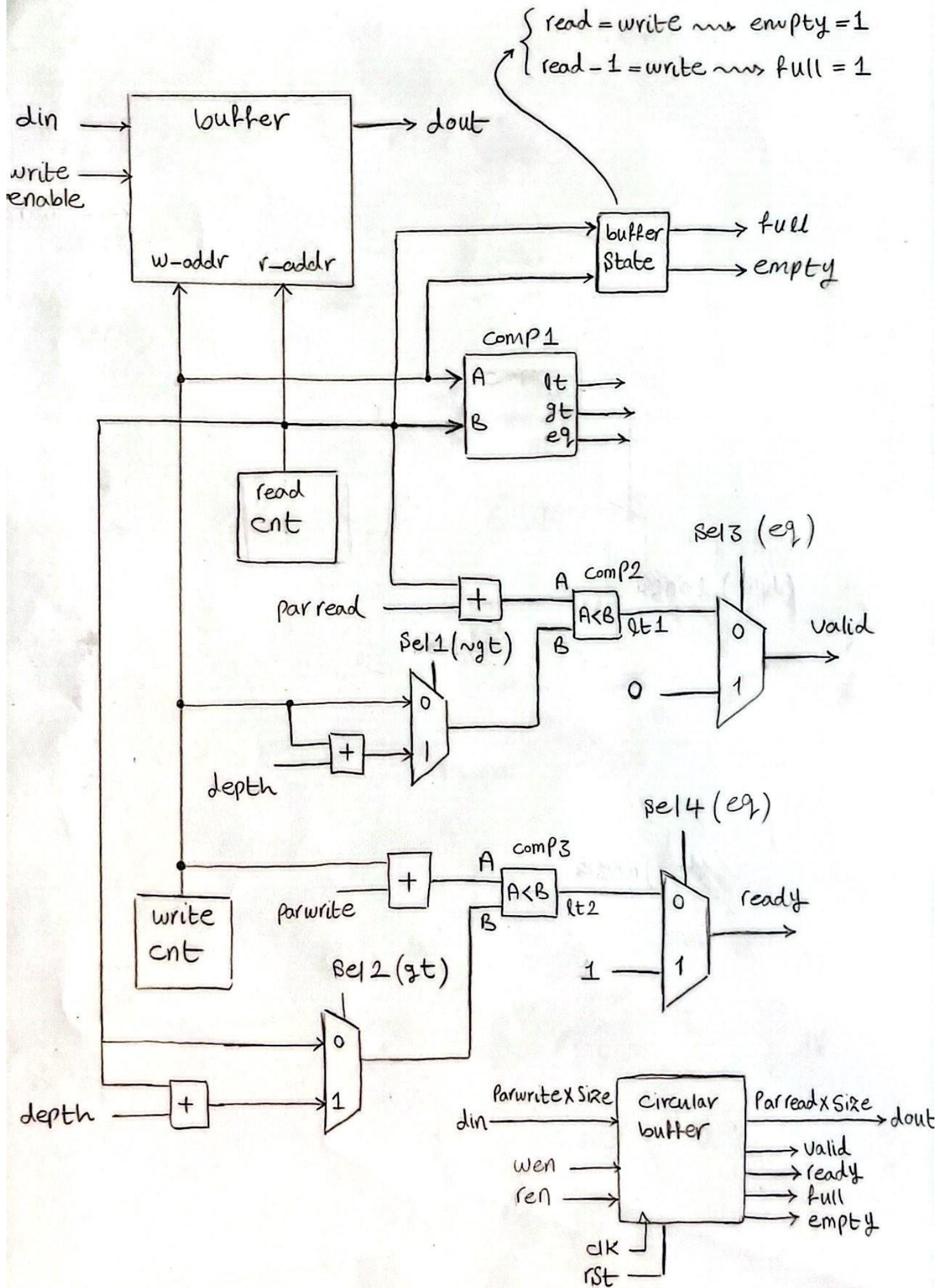




بحر باقر =



در ابتدا دو سیگنال `ready` و `valid` داریم که به ترتیب نمایانگر این هستند که آیا بافر ما حداقل فضای خالی برای نوشتن همزمان `par_write` تا داده را دارد یا نه و اینکه آیا در بافر ما به اندازه ی حداقل `par_read` تا داده برای خواندن وجود دارد یا نه. پس اگر بخواهیم به اندازه ی `par_write` تا داده در بافر خود بنویسیم باید چک کنیم که آیا `read_ptr` ما به اندازه ی حداقل `par_write` تا خانه جلو تر از `write_ptr` باشد یا نه. که اگر باشد یعنی ما توانایی نوشتن `par_write` تا داده را بدون `overwrite` شدن هیچ داده ی معتبری را داریم. البته هنگامی که `read_ptr` از `write_ptr` عقب تر باشد، برای اینکه چک کنیم که آیا `read_ptr` ما به اندازه ی حداقل `par_write` تا خانه جلو تر از `write_ptr` باشد کافی است که ابتدا `read_ptr` خود را با اندازه ی عمق بافر جمع کنیم و سپس آن را از `write_ptr` کم کنیم. همچنین برای سیگنال `valid` مشابه سیگنال `ready` فقط کافی است که چک کنیم که `write_ptr` حداقل به اندازه ی `par_read` تا خانه جلوتر از `read_ptr` باشد که در این صورت یعنی ما حداقل `par_read` تا داده ی خوانده نشده در حد فاصل `read_ptr` تا `write_ptr` داریم که مجازیم آنها را بخوانیم. همانند قبل اگر `write_ptr` عقب تر از `read_ptr` باشد کافی است که ابتدا `write_ptr` را با عمق بافر جمع کنیم و سپس `read_ptr` را از آن کم کنیم.

همچنین در آخر چک می کنیم که در صورت مساوی بودن `read_ptr` و `write_ptr` یعنی بافر ما خالی است و سیگنال `valid` را به صورت دستی 0 و سیگنال `ready` را به صورت دستی 1 می کنیم. که این یعنی بافر ما خالی است و می توانیم داده در بافر بنویسیم اما نمی توانیم از بافر داده بخوانیم چرا که بافر ما خالی است.

```

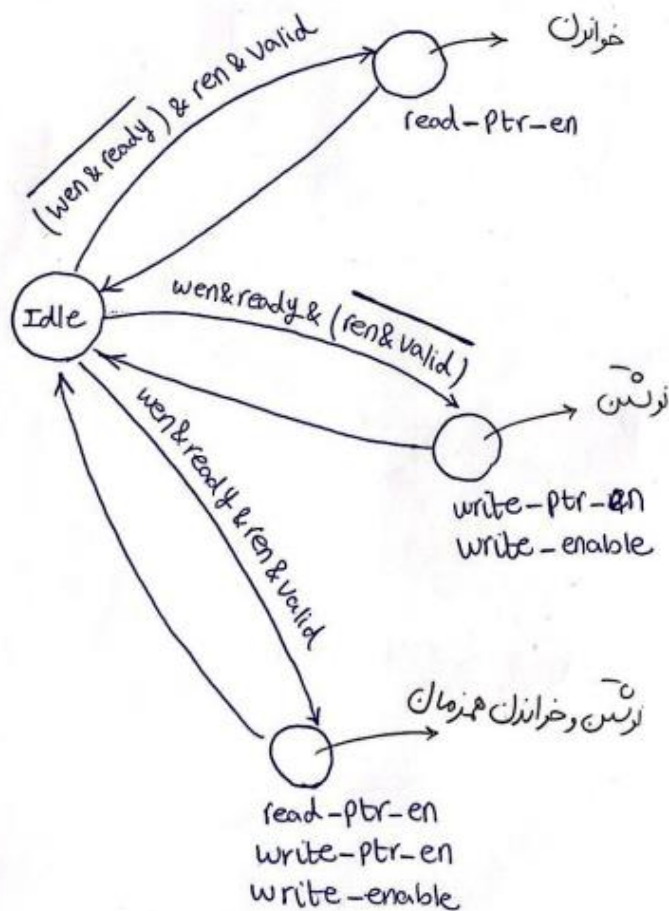
module buffer #(parameter PAR_READ, PAR_WRITE, DEPTH, SIZE)(input clk, rst, write_enable, input [3:0] waddr, raddr, input [(PAR_WRITE * SIZE)-1:0]
    reg [SIZE-1:0] memory [0:DEPTH-1];
    integer i;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            for (i = 0; i < DEPTH; i = i + 1)
                memory[i] <= 0;
        end else if (write_enable) begin
            for (i = 0; i < PAR_WRITE; i = i + 1) begin
                if ((waddr + i) < DEPTH)
                    memory[waddr + i] <= din[(i * SIZE) +: SIZE];
            end
        end
    end

    always @(*) begin
        for (i = 0; i < PAR_READ; i = i + 1) begin
            if ((raddr + i) < DEPTH)
                dout[(i * SIZE) +: SIZE] <= memory[raddr + i];
        end
    end
endmodule

```

با توجه به کد بالا، `write_ptr` و `read_ptr` نیز به ترتیب به عنوان `w_addr` و `r_addr` به بافر ما داده می شود که آدرس نوشتن و آدرس خواندن را تعیین می کنند. اما از آنجایی که ما باید توانایی نوشتن همزمان `par_write` تا داده را در بافر داشته باشیم پس ما باید به ازای هر `write_ptr` تا `write_ptr + par_write - 1` جلوی رویم و مثلاً داده ی `i` ام را در خانه ی متناظر آن بنویسیم. همچنین برای خواندن همزمان `par_read` تا داده لازم است که تا `read_ptr + par_read - 1` جلوی رویم و مثلاً داده ی `i` ام را از خانه ی متناظرش بخوانیم. که در این مسیر داده ما دو تا `counter` داریم که یکی `write_ptr` و دیگری `read_ptr` را می شمارد. به اینگونه که ما هر بار که به اندازه ی `par_write` تا داده در بافر خود می نویسیم `write_ptr` ما به علاوه ی `par_write` می شود و هر بار که به اندازه ی `par_read` تا داده از بافر خود می خوانیم، `read_ptr` ما به علاوه ی `par_read` می شود. البته باید حواسمان باشد از آنجا که بافر ما حلقوی است پس اگر `write_ptr + par_write` ما از سایز بافر بیشتر شد باید دوباره به اول بافر بیاید و از اول بافر شروع به شمارش کند. و همین قضیه نیز برای `read_ptr` وجود دارد.



توضیحات کنترلر : در این state digram ما ابتدا در idle هستیم و منتظریم که یک درخواستی از بیرون به ما برسد. به اینگونه که چک می کنیم که اگر $ren = 1$ بود و کسی از بیرون تقاضای خواندن از بافر را دارد ابتدا چک می کنیم که آیا سیگنال valid یک هست یا نه. یعنی در واقع ما توانایی خواندن همزمان par_read تا داده را داریم یا نه. که اگر این شروط

برقرار بود به استتیت مربوط به خواندن می رویم و read_ptr_en را یک می کنیم که در این صورت read_ptr به علاوه ی par_read می شود که در شکل زیر مشخص است :

```
module read_counter #(parameter PAR_READ, DEPTH) (input clk, rst, en,
    output reg [3:0] out);
    always@(posedge clk, posedge rst)begin
        if(rst == 1'b1)
            out <= 0;
        else if(en == 1'b1)
            out <= (out + PAR_READ) % DEPTH;
        end
    endmodule
```

همچنین ورودی های par_read و depth را به عنوان پارامتر به ماژول می دهیم تا در صورت نیاز بتوانیم مقادیر آنها را تعیین کنیم.

همچنین چک می کنیم که اگر $wen = 1$ بود و کسی از بیرون تقاضای نوشتن در بافر را دارد ابتدا چک می کنیم که آیا سیگنال ready یک هست یا نه. یعنی در واقع ما توانایی نوشتن همزمان par_write تا داده را داریم یا نه. که اگر این شروط برقرار بود به استتیت مربوط به نوشتن در بافر می رویم و write_ptr_en را یک می کنیم که در این صورت write_ptr به علاوه ی par_write می شود که در شکل زیر مشخص است :


```

module write_counter #(parameter PAR_WRITE, DEPTH) (input clk, rst, en,
    output reg [3:0] out);
    always@(posedge clk, posedge rst)begin
        if(rst == 1'b1)
            out <= 0;
        else if(en == 1'b1)
            out <= (out + PAR_WRITE) % DEPTH;
    end
endmodule

```

همچنین ورودی های `par_write` و `depth` را به عنوان پارامتر به ماژول می دهیم تا در صورت نیاز بتوانیم مقادیر آنها را تعیین کنیم. همچنین باقی مانده به `depth` را می گزاریم که پوینتر های ما به صورت حلقوی عمل کنند و در صورت رسیدن به انتهای بافر دوباره به اول بافر بیایند.

همچنین اگر هم `wen` و هم `ren` یک باشد یعنی کاربر ما قصد خواندن و نوشتن همزمان از بافر را دارد پس ابتدا چک می کنیم که سیگنال های `valid` و `ready` هر دو یک باشند که در این صورت از انجایی که نوشتن در بافر بر خواندن از بافر اولویت دارد ابتدا داده خواسته شده را در بافر می نویسیم و سپس داده خواسته شده را از بافر می خوانیم.